BTS SIO · DOCUMENTATION TECHNIQUE · PROCÉDURE D'INSTALLATION ET DE SÉCURISATION

Serveur GLPI sécurisé sous Debian 13

Mise en place d'un accès SSH durci, d'une protection contre le brute-force, de l'application de gestion de parc GLPI, d'une supervision Beszel et d'une sauvegarde automatisée vers un hébergement distant.

HYPERVISEUR	SYSTÈME	ADRESSE IP	PILE WEB	SGBD	APPLICATION
VMware (NAT)	Debian 13 Trixie	192.168.91.134	Apache 2 · PHP 8.4	MariaDB 11.8	GLPI 11.0.12

Poste d'administration
Windows · PowerShell · navigateur
→ SSH par clé `:22`
→ GLPI `:80`
→ Beszel `:8090`

VM Debian 13 — serveur (vm-debian · 192.168.91.134)

- **OpenSSH** — Authentification par clé — `22/tcp`
- **Fail2Ban** — Jail sshd + nftables — `3 échecs → 1 h`
- **GLPI 11** — Apache · PHP · MariaDB — `80/tcp`
- **Beszel** — Hub + Agent — `8090 · 45876`

InfinityFree
Hébergement distant
← archive .tar.gz
FTP passif `:21`
crontab · 3 h 00

1 Préparation du système

EN ROOT (SU -)

```
apt update && apt full-upgrade -y
apt install -y sudo curl wget nano open-vm-tools
usermod -aG sudo user
```

`apt update` rafraîchit la liste des paquets, `full-upgrade` applique toutes les mises à jour (y compris de sécurité).
`open-vm-tools` : intégration VMware (arrêt propre, synchronisation de l'heure). `usermod -aG sudo` donne les droits d'administration au compte `user`.

```
# Renouvellement de l'adresse IP (DHCP)
sudo ip link set ens33 up
sudo dhcpcd ens33
ip -br a ; ping -c 3 deb.debian.org
```

La carte réseau de la VM est en mode **NAT**. Une adresse en `169.254.x.x` signifie qu'aucun serveur DHCP n'a répondu.
`ip link set … up` active l'interface, `dhcpcd` demande une adresse, `ip -br a` l'affiche.

2 SSH — authentification par clé et durcissement

OPENSSH-SERVER

SUR LE POSTE D'ADMINISTRATION (PowerShell)

```
ssh-keygen
type $env:USERPROFILE\.ssh\<nom_de_la_cle>.pub | ssh user@192.168.91.134 "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"

Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent ; ssh-add
```

`ssh-keygen` crée une paire de clés : la clé **privée** reste sur le poste, la clé **publique** (`.pub`) est copiée sur le serveur.
La 2e commande ajoute la clé publique dans `authorized_keys` avec les droits stricts exigés par SSH (700 / 600).
`ssh-agent` + `ssh-add` mémorisent la passphrase de la clé. La clé privée n'est jamais copiée ni publiée.

SUR LE SERVEUR /etc/ssh/sshd_config.d/99-hardening.conf

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowUsers user
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
```

- **Root interdit** en SSH : administration via `sudo`.
- **Mots de passe désactivés** : seule la clé est acceptée, le brute-force devient inopérant.
- `AllowUsers` : liste blanche des comptes autorisés.
- `MaxAuthTries` / `LoginGraceTime` : 3 essais, 30 s pour s'authentifier.
- `X11Forwarding no` : fonction graphique inutile sur un serveur.
- `ClientAlive…` : coupure des sessions inactives (~10 min).

Le dossier `sshd_config.d` évite de modifier le fichier d'origine.

```
sudo sshd -t && sudo systemctl restart ssh
sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthentication|allowusers'
```

`sshd -t` vérifie la syntaxe avant de redémarrer le service. `sshd -T` affiche la configuration réellement appliquée.

Contrôle : `ssh user@192.168.91.134` se connecte par clé, `ssh root@192.168.91.134` renvoie `Permission denied (publickey)`.

3 Fail2Ban — protection du service SSHD

FAIL2BAN · NFTABLES

Fail2Ban analyse les journaux du service SSH et bannit temporairement, via le pare-feu, toute adresse IP qui accumule les échecs d'authentification.

```
sudo apt install -y fail2ban python3-systemd nftables
sudo nano /etc/fail2ban/jail.local
```

`python3-systemd` permet de lire le journal systemd ; `nftables` est le pare-feu qui applique les blocages.

```
[DEFAULT]
bantime   = 1h
findtime  = 10m
maxretry  = 3
backend   = systemd
banaction = nftables-multiport
ignoreip  = 127.0.0.1/8 ::1

[sshd]
enabled      = true
port         = ssh
journalmatch = _SYSTEMD_UNIT=ssh.service
```

- **3 échecs en 10 min** → bannissement de **1 h**.
- `backend = systemd` : Debian 13 n'utilise plus `/var/log/auth.log`.
- `ignoreip` : la machine locale n'est jamais bannie.
- `journalmatch` : sous Debian 13 (OpenSSH 10), les échecs sont journalisés par le processus `sshd-session` ; le filtre lit donc tout le service `ssh.service`.
- `jail.local` surcharge `jail.conf` sans le modifier.

```
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo nft list ruleset | grep -A3 f2b
sudo fail2ban-client set sshd unbanip 192.168.91.1
```

`enable --now` démarre le service et l'active au démarrage. `status sshd` affiche les échecs et les IP bannies.
`nft list ruleset` montre la règle de rejet créée sur le port 22 ; `unbanip` lève un bannissement.

Test : après 4 tentatives `ssh root@…`, l'IP du poste (`192.168.91.1` , passerelle NAT VMware) apparaît dans *Banned IP list* et le port 22 ne répond plus.

4 GLPI — Apache 2, MariaDB et PHP

GLPI 11.0.12

```
sudo apt install -y apache2 mariadb-server php libapache2-mod-php \
  php-{mysql,mbstring,curl,gd,xml,intl,ldap,apcu,zip,bz2,bcmath}
sudo mariadb-secure-installation
sudo mariadb-tzinfo-to-sql /usr/share/zoneinfo | sudo mariadb mysql
```

Installation de la pile LAMP et des extensions PHP requises par GLPI.

`mariadb-secure-installation` supprime les comptes anonymes, la base de test et l'accès root distant. La 3e commande charge les fuseaux horaires (exigé par GLPI).

BASE DE DONNÉES (sudo mariadb)

```
CREATE DATABASE glpi CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'glpi'@'localhost' IDENTIFIED BY '<MOT_DE_PASSE>';
GRANT ALL PRIVILEGES ON glpi.* TO 'glpi'@'localhost';
GRANT SELECT ON mysql.time_zone_name TO 'glpi'@'localhost';
FLUSH PRIVILEGES;
```

Base `glpi` en UTF-8 complet. Utilisateur dédié, connectable uniquement en local, avec des droits limités à sa base : **principe du moindre privilège**.

FICHIERS DE L'APPLICATION

```
wget https://github.com/glpi-project/glpi/releases/download/11.0.12/glpi-
11.0.12.tgz
sudo tar -xzf glpi-11.0.12.tgz -C /var/www/
sudo mv /var/www/glpi/config /etc/glpi
sudo mv /var/www/glpi/files  /var/lib/glpi
sudo mkdir -p /var/log/glpi
sudo chown -R www-data:www-data /var/www/glpi /etc/glpi /var/lib/glpi
/var/log/glpi
```

Téléchargement et extraction de GLPI. La configuration, les données et les journaux sont **déplacés hors du dossier web** (bonne pratique de sécurité).

`chown` donne la propriété des fichiers au compte d'Apache, `www-data` .

```
// /var/www/glpi/inc/downstream.php
<?php
define('GLPI_CONFIG_DIR', '/etc/glpi/');
if (file_exists(GLPI_CONFIG_DIR . '/local_define.php')) {
    require_once GLPI_CONFIG_DIR . '/local_define.php';
}
// /etc/glpi/local_define.php
<?php
define('GLPI_VAR_DIR', '/var/lib/glpi');
define('GLPI_LOG_DIR', '/var/log/glpi');
```

`downstream.php` indique à GLPI où se trouve sa configuration (`/etc/glpi`).

`local_define.php` indique l'emplacement des données (`/var/lib/glpi`) et des journaux (`/var/log/glpi`).

HÔTE VIRTUEL APACHE /etc/apache2/sites-available/glpi.conf

```
<VirtualHost *:80>
    ServerName glpi.lan
    DocumentRoot /var/www/glpi/public
    <Directory /var/www/glpi/public>
        Require all granted
        RewriteEngine On
        RewriteCond %{HTTP:Authorization} ^(.+)$
        RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
        RewriteCond %{REQUEST_FILENAME} !-f
        RewriteRule ^(.*)$ index.php [QSA,L]
    </Directory>
    ErrorLog  ${APACHE_LOG_DIR}/glpi_error.log
    CustomLog ${APACHE_LOG_DIR}/glpi_access.log combined
</VirtualHost>
```

- `DocumentRoot …/public` : seul le dossier `public` est exposé, le code et les fichiers sensibles restent inaccessibles.
- Les 2 premières règles transmettent l'en-tête *Authorization* (API et agents d'inventaire).
- Les 2 suivantes redirigent toute requête vers `index.php` , le routeur de GLPI.
- Journaux d'erreurs et d'accès dédiés au site.

```
sudo sed -i 's/^;\?session.cookie_httponly =.*/session.cookie_httponly = On/'
/etc/php/8.4/apache2/php.ini
sudo a2enmod rewrite
sudo a2dissite 000-default && sudo a2ensite glpi
sudo apache2ctl configtest && sudo systemctl restart apache2
sudo rm /var/www/glpi/install/install.php
```

`cookie_httponly` empêche le JavaScript de lire le cookie de session (protection XSS).

`a2enmod` active la réécriture d'URL, `a2dissite` / `a2ensite` remplacent le site par défaut par GLPI, `configtest` valide la syntaxe.

Après l'assistant web, suppression du script d'installation.

Après installation : changement des mots de passe des comptes par défaut `glpi` , `tech` , `normal` , `post-only` et désactivation des données de démonstration.

5 Beszel — supervision du serveur

HUB :8090 · AGENT :45876

Le **Hub** est l'interface web qui stocke l'historique et affiche les graphiques ; l'**Agent** mesure CPU, mémoire, disque, réseau et services, puis transmet les données au Hub.

```
curl -sL https://get.beszel.dev/hub -o /tmp/install-hub.sh
chmod +x /tmp/install-hub.sh && sudo /tmp/install-hub.sh
sudo ss -tlnp | grep 8090

curl -sL https://get.beszel.dev -o /tmp/install-agent.sh
sudo chmod +x /tmp/install-agent.sh
sudo /tmp/install-agent.sh -p 45876 -k "<CLE_PUBLIQUE_HUB>" \
  -t "<TOKEN>" -url "http://192.168.91.134:8090"
systemctl status beszel-agent --no-pager
```

Le script officiel télécharge le Hub et crée un service systemd. `ss -tlnp` vérifie l'écoute sur le port 8090.

Dans l'interface : création du compte administrateur, puis *Ajouter un système* → la commande de l'agent est générée avec la **clé publique** et le **token** d'enregistrement.

L'agent s'exécute sous un compte dédié `beszel` , non root.

6 Sauvegarde automatisée vers InfinityFree

BASH · MARIADB-DUMP · TAR · FTP

```
sudo -i
apt install -y ftp
mkdir -p /root/scripts /root/backups
```

Les scripts s'exécutent en **root** (accès à la base et aux fichiers). Client FTP, dossier des scripts et dossier des archives locales.

```
# backupGLPI.sh — script principal
#!/bin/bash
cd "$(dirname "$0")" || exit 1
laDate=$(date +%Y.%m.%d_%Hh%M)
bash dumpsql.sh $laDate
bash targz.sh   $laDate
bash sendftp.sh $laDate
find /root/backups -name '*_glpi.tar.gz' -mtime +7 -delete
```

`cd dirname` : se place dans le dossier du script (indispensable sous cron). Une date unique nomme tous les fichiers. `find … -mtime +7 -delete` : rotation, suppression des archives locales de plus de 7 jours.

```
# dumpsql.sh — export de la base
#!/bin/bash
echo "*1* DUMPSQL de la base GLPI ***"
mariadb-dump --single-transaction glpi > /tmp/$1_glpi.sql
```

Export SQL complet de la base. `--single-transaction` : copie cohérente sans bloquer GLPI. Connexion root par socket Unix : **aucun mot de passe en clair**.

```
# targz.sh — création de l'archive
#!/bin/bash
archive=$1_glpi.tar.gz
echo "*2* Archivage de GLPI ***"
tar -czf /root/backups/$archive \
  --exclude=var/lib/glpi/_cache \
  --exclude=var/lib/glpi/_sessions \
  --exclude=var/lib/glpi/_tmp \
  -C / etc/glpi var/lib/glpi tmp/$1_glpi.sql
rm /tmp/$1_glpi.sql
```

Archive compressée de la configuration, des données et du dump SQL ; les dossiers temporaires sont exclus. Le code applicatif n'est pas inclus (re-téléchargeable).

```
# sendftp.sh — envoi sur InfinityFree
#!/bin/bash
archive=$1_glpi.tar.gz
FTP_USER="<IDENTIFIANT_FTP>"
FTP_PASS="<MOT_DE_PASSE_FTP>"
taille=$(stat -c %s /root/backups/$archive)
if [ "$taille" -gt 10000000 ]; then
  echo "ERREUR : archive > 10 Mo"; exit 1
fi
echo "*3* Envoi sur le FTP InfinityFree ***"
ftp -inpv ftpupload.net << EOF
user $FTP_USER $FTP_PASS
binary
mkdir backups
cd backups
put /root/backups/$archive $archive
bye
EOF
```

- Contrôle de la **limite de 10 Mo** par fichier imposée par InfinityFree.
- `-i` sans confirmation, `-n` connexion manuelle, `-p` mode **passif** (traversée du NAT), `-v` affichage des réponses.
- `binary` : transfert sans altération de l'archive.
- Dossier `backups` hors de `htdocs` : non accessible publiquement.

```
chmod 700 /root/scripts/*.sh
/root/scripts/backupGLPI.sh
```

`chmod 700` : scripts lisibles et exécutables par root uniquement (identifiants FTP).

CODE FTP	SIGNIFICATION
230	Authentification réussie
226	Transfert terminé avec succès
550	Dossier déjà existant (sans conséquence)

7 Planification avec crontab

TOUS LES JOURS À 3 H 00

```
crontab -e
0 3 * * * /root/scripts/backupGLPI.sh >> /var/log/backupGLPI.log 2>&1
crontab -l
tail -20 /var/log/backupGLPI.log
```

Champs : **minute · heure · jour du mois · mois · jour de la semaine** → `0 3 * * *` = chaque nuit à 3 h 00.

`>>` ajoute la sortie au journal, `2>&1` y inclut les erreurs. `crontab -l` liste les tâches.

```
tar -xzf <date>_glpi.tar.gz -C /
mariadb glpi < /tmp/<date>_glpi.sql
chown -R www-data:www-data /etc/glpi /var/lib/glpi
```

Restauration : extraction des fichiers à leur emplacement d'origine, réimport de la base, puis rétablissement des droits d'Apache.

8 Publication du dépôt sur GitHub

GIT · POWERSHELL

PREMIER ENVOI (dépôt distant déjà créé)

```
cd "<dossier du dépôt>"
git init -b main
git remote add origin https://github.com/<utilisateur>/Serveur-BTS-2026.git
git fetch origin
git reset origin/main
git add -A
git add --chmod=+x "Scripts/sauvegarde/*.sh"
git commit -m "Serveur BTS 2026"
git push -u origin main
```

`init` crée le dépôt local, `remote add` le relie à GitHub. `fetch` + `reset origin/main` reprennent l'historique distant sans toucher aux fichiers locaux. `--chmod=+x` conserve le droit d'exécution des scripts (non géré par Windows). `push -u` envoie la branche et mémorise la destination.

REMPLACER LE CONTENU DISTANT

```
git push --force origin main
```

Écrase l'historique de GitHub par l'historique local : à réserver au remplacement complet d'un dépôt (les commits distants absents en local sont perdus).

MISES À JOUR SUIVANTES

```
git status
git add -A
git commit -m "Mise à jour de la procédure"
git push
```

`status` liste les fichiers modifiés ; chaque modification est enregistrée par un commit puis envoyée.

Avant tout push : aucun mot de passe, token, clé privée ou identifiant réel dans les fichiers (valeurs remplacées par des marqueurs `<…>`).

✓ Vérifications

SERVICE	COMMANDE / CONTRÔLE	RÉSULTAT ATTENDU
SSH	`sudo sshd -T` · `ssh root@192.168.91.134`	permitrootlogin no · passwordauthentication no · accès root refusé
Fail2Ban	`sudo fail2ban-client status sshd` · `sudo nft list ruleset`	Échecs comptabilisés, IP bannie, règle `reject` sur le port 22
GLPI	`http://192.168.91.134`	Tableau de bord sans alerte de sécurité
Beszel	`http://192.168.91.134:8090`	Système *vm-glpi* au vert, métriques en temps réel
Sauvegarde	`crontab -l` · `/var/log/backupGLPI.log`	`226 File successfully transferred`, archive présente dans `/backups`